Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel — обработка символьных данных»

Выполнил:
Студент 4-го курса
Группы АС-66
Савинец М. Д.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы

Изучение программной модели микропроцессора и обработка символьных данных с использованием регистров, оперативной памяти и отдельных ассемблерных процедур.

Задание варианта 6

Ввести непустой текст длиной не более 100 символов. Признак конца ввода — точка, которая в текст не входит. Проверить, содержит ли текст равное количество прописных латинских и прописных русских букв. Если условие выполнено, заменить каждую строчную латинскую букву прописной, а каждую прописную латинскую — строчной. При невыполнении условия вывести сообщение. Некорректный ввод не преобразовывать.

Проверка условия и преобразование выполняются отдельными процедурами. Ввод, проверка, обработка и вывод следуют последовательно. Дополнительная память, зависящая от длины текста, и изменяемые глобальные переменные не используются. Вариант 6 не имеет отметки «с.к.», поэтому строковые команды с REP не обязательны.

Представление данных и алгоритм

Программа предназначена для macOS на Apple Silicon, архитектура ARM64. Файл main.cpp содержит ввод, проверку допустимости текста, последовательный вызов процедур и вывод. В файле lab2_mac.S реализованы ассемблерные процедуры check_condition и transform_text. Сборка выполняется Apple Clang, стандарт C++17. Ввод и вывод используют UTF-8; внутри текст хранится в фиксированном массиве std::array<char32_t, 101>. Один элемент занимает 4 байта и хранит одну кодовую точку Unicode. Длина передаётся явно. Выполнение на Mac согласовано с преподавателем.

check_condition подсчитывает буквы A–Z и русские А–Я, отдельно учитывая Ё. Результат — 1, если счётчики равны, и 0 в противном случае. Равенство 0 = 0 удовлетворяет условию. transform_text проверяет диапазоны A–Z и a–z и переключает бит 0x20 командой EOR. Русские буквы и остальные символы сохраняются. Обе процедуры работают за O(n), дополнительная память постоянна.

Регистры и вызовы

Используется соглашение о вызовах Apple ARM64. X0 получает адрес массива, X1 — длину. В check_condition X2 и X3 хранят счётчики, W4 — код символа, W5 — промежуточное значение. В transform_text используются W2 и W3. Указатель увеличивается на 4 байта. Результат проверки возвращается через W0 командой CSET. Процедуры используют только caller-saved регистры, не вызывают другие функции и завершаются RET. Объявления extern "C" в lab2.h связывают C++ с ассемблерными процедурами.

Текст программы

Ниже приведены полный код программы для Mac ARM64, объявления процедур и тестовый стенд. Запуск выполняется сценарием run_mac.command. Режим --demo показывает примеры, --test запускает проверки, запуск без аргументов позволяет ввести текст. Динамические строки и копии данных используются только в тестовых режимах.

lab2.h

```
#pragma once
#include <cstddef>
extern "C" int check_condition(const char32_t* text, std::size_t n);
extern "C" void transform_text(char32_t* text, std::size_t n);
```

lab2_mac.S

```
// Apple Silicon adaptation, same C ABI and UTF-32 representation.
.text
.p2align 2
.globl _check_condition
_check_condition:
    mov x2, #0
    mov x3, #0
    cbz x1, 4f
1:  ldr w4, [x0], #4
    sub w5, w4, #65
    cmp w5, #25
    b.hi 2f
    add x2, x2, #1
    b 3f
2:  cmp w4, #0x401
    b.eq 5f
    sub w5, w4, #0x410
    cmp w5, #31
    b.hi 3f
5:  add x3, x3, #1
3:  subs x1, x1, #1
    b.ne 1b
4:  cmp x2, x3
    cset w0, eq
    ret
.p2align 2
.globl _transform_text
_transform_text:
    cbz x1, 4f
1:  ldr w2, [x0]
    sub w3, w2, #65
    cmp w3, #25
    b.ls 2f
    sub w3, w2, #97
    cmp w3, #25
    b.hi 3f
2:  eor w2, w2, #0x20
    str w2, [x0]
3:  add x0, x0, #4
    subs x1, x1, #1
    b.ne 1b
4:  ret
```

main.cpp

```
#include "lab2.h"
#include <algorithm>
#include <stdexcept>
#include <string>
#include <array>
```

```
#include <iostream>
#include <sstream>
#include <random>

constexpr std::size_t MAX_TEXT = 100;
struct Input {
    std::array<char32_t, MAX_TEXT + 1> text{};
    std::size_t n = 0;
};

// Decode directly into a fixed buffer; no text-sized auxiliary buffer.
Input read_text(std::istream& in) {
    Input result;
    for (;;) {
        int first = in.get();
        if (first == EOF) throw std::runtime_error("Missing final period");
        if (first == '.') break;
        unsigned char c = static_cast<unsigned char>(first);
        unsigned extra = 0;
        char32_t cp = 0, minimum = 0;
        if (c < 0x80) cp = c;
        else if (c >= 0xC2 && c <= 0xDF) {
            cp = c & 31; extra = 1; minimum = 0x80;
        } else if (c >= 0xE0 && c <= 0xEF) {
            cp = c & 15; extra = 2; minimum = 0x800;
        } else if (c >= 0xF0 && c <= 0xF4) {
            cp = c & 7; extra = 3; minimum = 0x10000;
        } else throw std::runtime_error("Invalid UTF-8");
        for (unsigned j = 0; j < extra; ++j) {
            int next = in.get();
            if (next == EOF || (next & 0xC0) != 0x80)
                throw std::runtime_error("Invalid UTF-8 continuation");
            cp = (cp << 6) | (next & 63);
        }
        if (cp < minimum || cp > 0x10FFFF || (cp >= 0xD800 && cp <= 0xDFFF))
            throw std::runtime_error("Invalid Unicode code point");
        if (result.n == MAX_TEXT) throw std::runtime_error("More than 100 characters");
        result.text[result.n++] = cp;
    }
    if (!result.n) throw std::runtime_error("Empty text");
    return result;
}

void print_text(const Input& input) {
    // Fixed-size UTF-8 scratch for one character, not the entire text.
    for (std::size_t i = 0; i < input.n; ++i) {
        const auto cp = static_cast<unsigned>(input.text[i]);
        char b[4]; int n;
        if (cp < 0x80) { b[0] = static_cast<char>(cp); n = 1; }
        else if (cp < 0x800) {
            b[0] = static_cast<char>(0xC0 | (cp >> 6));
            b[1] = static_cast<char>(0x80 | (cp & 63)); n = 2;
        } else if (cp < 0x10000) {
            b[0] = static_cast<char>(0xE0 | (cp >> 12));
            b[1] = static_cast<char>(0x80 | ((cp >> 6) & 63));
            b[2] = static_cast<char>(0x80 | (cp & 63)); n = 3;
        } else {
            b[0] = static_cast<char>(0xF0 | (cp >> 18));
            b[1] = static_cast<char>(0x80 | ((cp >> 12) & 63));
            b[2] = static_cast<char>(0x80 | ((cp >> 6) & 63));
            b[3] = static_cast<char>(0x80 | (cp & 63)); n = 4;
        }
        std::cout.write(b, n);
    }
}
```

```
void run(std::istream& in) {
    std::cout << "Enter 1..100 characters, end with a period.\n"
              << "Condition: uppercase Latin count equals uppercase Russian count (including Yo).\n"
              << "If true: swap case of Latin letters.\n";
    auto input = read_text(in);
    std::cout << "Original: "; print_text(input); std::cout << '\n';
    const bool ok = check_condition(input.text.data(), input.n) != 0;
    std::cout << "Condition: " << (ok ? "TRUE" : "FALSE") << '\n';
    if (ok) transform_text(input.text.data(), input.n);
    else std::cout << "Transformation skipped.\n";
    std::cout << "Result: "; print_text(input); std::cout << '\n';
}

void require(bool ok) {
    if (!ok) throw std::runtime_error("Test failed");
}
void tests() {
    const std::pair<std::u32string, bool> cases[] = {
        {U"AaБб", true}, {U"ABAЯ", true}, {U"AЁ", true},
        {U"ABЁ", false}, {U"ABCA", false}, {U"abc", true},
        {U"123 !", true}, {U"Ё", false}, {U"ё", true},
        {U"AZazAЯЁaяё", false}, {U"@AZ[ `az{", false}
    };
    for (const auto& item : cases) {
        std::array<char32_t, 103> b{};
        b.front() = b.back() = U'@';
        std::copy(item.first.begin(), item.first.end(), b.begin() + 1);
        require((check_condition(b.data() + 1, item.first.size()) != 0) == item.second);
        transform_text(b.data() + 1, item.first.size());
        for (std::size_t i = 0; i < item.first.size(); ++i) {
            auto c = item.first[i];
            auto expected = c >= U'A' && c <= U'Z' ? c + 32 :
                            c >= U'a' && c <= U'z' ? c - 32 : c;
            require(b[i + 1] == expected);
        }
        require(b.front() == U'@' && b.back() == U'@');
    }
    std::mt19937 rng(206);
    for (std::size_t n = 0; n <= 100; ++n) {
        for (int round = 0; round < 50; ++round) {
            std::array<char32_t, 102> b{};
            b[0] = b[n + 1] = U'@';
            unsigned latin = 0, russian = 0;
            for (std::size_t i = 1; i <= n; ++i) {
                b[i] = static_cast<char32_t>(rng() % 0x500);
                latin += b[i] >= U'A' && b[i] <= U'Z';
                russian += (b[i] >= U'A' && b[i] <= U'Я') || b[i] == U'Ё';
            }
            require((check_condition(b.data() + 1, n) != 0) == (latin == russian));
            auto original = b;
            transform_text(b.data() + 1, n);
            for (std::size_t i = 1; i <= n; ++i) {
                auto c = original[i];
                require(b[i] == (c >= U'A' && c <= U'Z' ? c + 32 :
                                 c >= U'a' && c <= U'z' ? c - 32 : c));
            }
            require(b[0] == U'@' && b[n + 1] == U'@');
        }
    }
    for (const auto& s : {".", "abc", "\x80.", "\xC0\xAF.", "\xED\xA0\x80.", "\xF4\x90\x80\x80."}) {
        std::istringstream in(s); bool rejected = false;
        try { (void)read_text(in); } catch (const std::runtime_error&) { rejected = true; }
        require(rejected);
    }
    std::istringstream limit(std::string(100, 'a') + ".");
    require(read_text(limit).n == 100);
```

```
    std::istringstream over(std::string(101, 'a') + ".");
    bool rejected = false;
    try { (void)read_text(over); } catch (const std::runtime_error&) { rejected = true; }
    require(rejected);
    std::istringstream stop("АБ.rest"); require(read_text(stop).n == 2);
    std::cout << "PASS: 11 known cases, including capital Yo and Latin boundaries\n"
              << "PASS: 5050 randomized cases, lengths 0..100, boundary guards\n"
              << "PASS: empty / missing period / invalid UTF-8 / 100 and 101 characters\n";
}

int main(int argc, char** argv) {
    try {
        std::cout << "Lab 2 | Variant 6\n";
        const std::string mode = argc > 1 ? argv[1] : "";
        if (mode == "--test") tests();
        else if (mode == "--demo") {
            for (const char* s : {"АаБб.", "ABCA.", "АЁ.", "abc 123.", "."}) {
                std::cout << "\nInput: " << s << '\n';
                std::istringstream in(s);
                try { run(in); } catch (const std::exception& e) {
                    std::cout << "Rejected: " << e.what() << '\n';
                }
            }
        } else if (mode.empty()) run(std::cin);
        else throw std::runtime_error("Usage: lab2 [--test | --demo]");
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << '\n'; return 1;
    }
}
```

Результаты выполнения

На рисунке показан скриншот просмотра сохранённого вывода реально выполненного режима --demo и проверок --test на Mac ARM64. Случаи АаБб и АЁ удовлетворяют условию, ABCA — не удовлетворяет. В строке abc 123 оба счётчика равны нулю, поэтому результат ABC 123.

Лабораторная 2 — сохранённый вывод запуска на Mac ARM64

```
Lab 2 | Variant 6

Input: АаБб.
Enter 1..100 characters, end with a period.
Condition: uppercase Latin count equals uppercase Russian count (including Yo).
If true: swap case of Latin letters.
Original: АаБб
Condition: TRUE
Result: аАбб

Input: ABCA.
Enter 1..100 characters, end with a period.
Condition: uppercase Latin count equals uppercase Russian count (including Yo).
If true: swap case of Latin letters.
Original: ABCA
Condition: FALSE
Transformation skipped.
Result: ABCA

Input: АЁ.
Enter 1..100 characters, end with a period.
Condition: uppercase Latin count equals uppercase Russian count (including Yo).
If true: swap case of Latin letters.
Original: АЁ
Condition: TRUE
Result: аё

Lab 2 | Variant 6
PASS: 11 known cases, including capital Yo and Latin boundaries
PASS: 5050 randomized cases, lengths 0..100, boundary guards
PASS: empty / missing period / invalid UTF-8 / 100 and 101 characters
```

Рисунок 1 — Просмотр сохранённых результатов запуска Mac ARM64

Проверка правильности

Пройдены 11 контрольных примеров и 5050 случайных проверок для длин от 0 до 100 включительно. Отдельно проверены буква Ё, границы диапазонов A–Z и a–z, неизменность русских букв и символов вне латинских диапазонов, а также контрольные элементы по краям буфера. Нулевая длина проверяется на уровне процедур; пользовательский интерфейс пустой текст отклоняет.

Проверены ввод ровно 100 символов, отклонение 101 символа, пустой текст, отсутствие точки, некорректный UTF-8 и остановка на первой точке. Дополнительный запуск с AddressSanitizer и UndefinedBehaviorSanitizer завершился успешно. Ассемблерные инструкции этими средствами не инструментируются; записи вне массива контролируются также защитными элементами.

Все приведённые результаты получены при реальном запуске программы на Mac ARM64. Журналы выполнения и скриншот сохранены вместе с проектом в папке results.

Вывод

Реализовал проверку условия и смену регистра латинских букв отдельными ассемблерными процедурами ARM64. Закрепил работу с адресами памяти, диапазонами кодов символов, счётчиками, условными переходами и вызовом ассемблера из C++. Программа для Mac прошла контрольные и случайные проверки. Ограничение длины и завершение ввода точкой реализованы явно.